

PROGRAMAÇÃO FUNCIONAL

AMBIENTAÇÃO E EXPERIMENTAÇÃO COM HASKELL

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

PREPARAR O ECOSSISTEMA

- (Windows) Instalar o GHCup: Abra o Powershell e copie o seguinte comando:

```
Set-ExecutionPolicy Bypass -Scope Process -Force;  
[System.Net.ServicePointManager]::SecurityProtocol =  
[System.Net.ServicePointManager]::SecurityProtocol -bor  
3072; try { & ([ScriptBlock]::Create((Invoke-WebRequest  
https://www.haskell.org/ghcup/sh/bootstrap-haskell.ps1 -  
UseBasicParsing))) -Interactive -DisableCurl } catch { Write-  
Error S_ }
```

Aceite todos os adicionais.

PREPARAR O ECOSSISTEMA

- (Linux) Instalar o GHCup: Abra o Terminal e copie o seguinte comando:

```
curl --proto '=https' --tlsv1.2 -sSf https://get-ghcup.haskell.org | sh
```

PREPARAR O ECOSSISTEMA

- Agora basta digitar “ghci” no PowerShell/Terminal que o ambiente será iniciado.

USANDO O TERMINAL

- O comando `cd` te permite mudar de diretório:
 - `cd nomeDoDiretorio` –Avança
 - `cd ..` – Retorna

USANDO O TERMINAL

- O comando `ls` lista todos os diretórios dentro do seu diretório atual.

USANDO O TERMINAL

- Também é possível clicar com o botão direito em uma pasta e abrir o terminal diretamente naquele diretório.

AVALIAÇÃO DIRETA

- Digite expressões (ex: $2 + 2$ ou reverse "Haskell") e veja o resultado de imediato. Excelente para prototipação e verificação rápida de ideias sem precisar compilar um projeto inteiro.

INSPEÇÃO DE TIPOS

No Haskell, os tipos são seus guias. Use o comando `:t expressão` (ou `:type`) no terminal para fazer o compilador revelar o tipo deduzido de qualquer código que você digitar.

CARREGAMENTO

- Escreva lógicas complexas em um arquivo de texto (ex: script.hs). No terminal, use o comando `:l script.hs` (load) para importar suas funções e brincar com elas interativamente.

CARREGAMENTO

- Para gerar arquivos executaveis usamos a seguinte linha de comando, fora do ambiente ghci:
- `ghc -o nomeDoExe nomeDoCodFonte.hs`

Em seguida ./nomeDoExe

*O main deve ser definido

ANATOMIA DO SEU PRIMEIRO CÓDIGO

Funções Matemáticas Puras

A sintaxe é limpa e livre de ruídos estruturais.

Parecem equações:

1. $\text{dobro} :: \text{Int} \rightarrow \text{Int}$

2. $\text{dobro } x = x * 2$

Não usamos a palavra "return". A função é a expressão mapeada.

EXEMPLOS DE CODIGO

- 1.-- Cálculo simples: Área de um círculo
- 2.
- 3.areaCirculo :: Double -> Double
- 4.areaCirculo raio = pi * raio ^ 2

EXEMPLO

- 1.-- 2. Cálculo com recursão e pattern matching
- 2.fatorial :: Integer -> Integer
- 3.fatorial 0 = 1
- 4.fatorial n = n * fatorial (n - 1)

EXEMPLO

- 1.-- 3. Manipulação de lista:
- 2.-- Soma de todos os elementos
- 3.
4. $\text{somaDaLista} :: [\text{Int}] \rightarrow \text{Int}$
5. $\text{somaDaLista} [] = 0$
6. $\text{somaDaLista} (x : xs) = x + \text{somaDaLista } xs$

EXEMPLO

```
1.-- 4. Função principal (Ponto de entrada)
2.main :: IO ()
3.main = do
4.    print $ areaCirculo 5.0
5.    print $ somaDaLista [1..10]
6.    print $ fatorial 6
```


DESAFIO

1. Tente criar uma função que soma dois numeros.
2. Tente criar uma função para calcular Fibonacci.

DÚVIDAS???